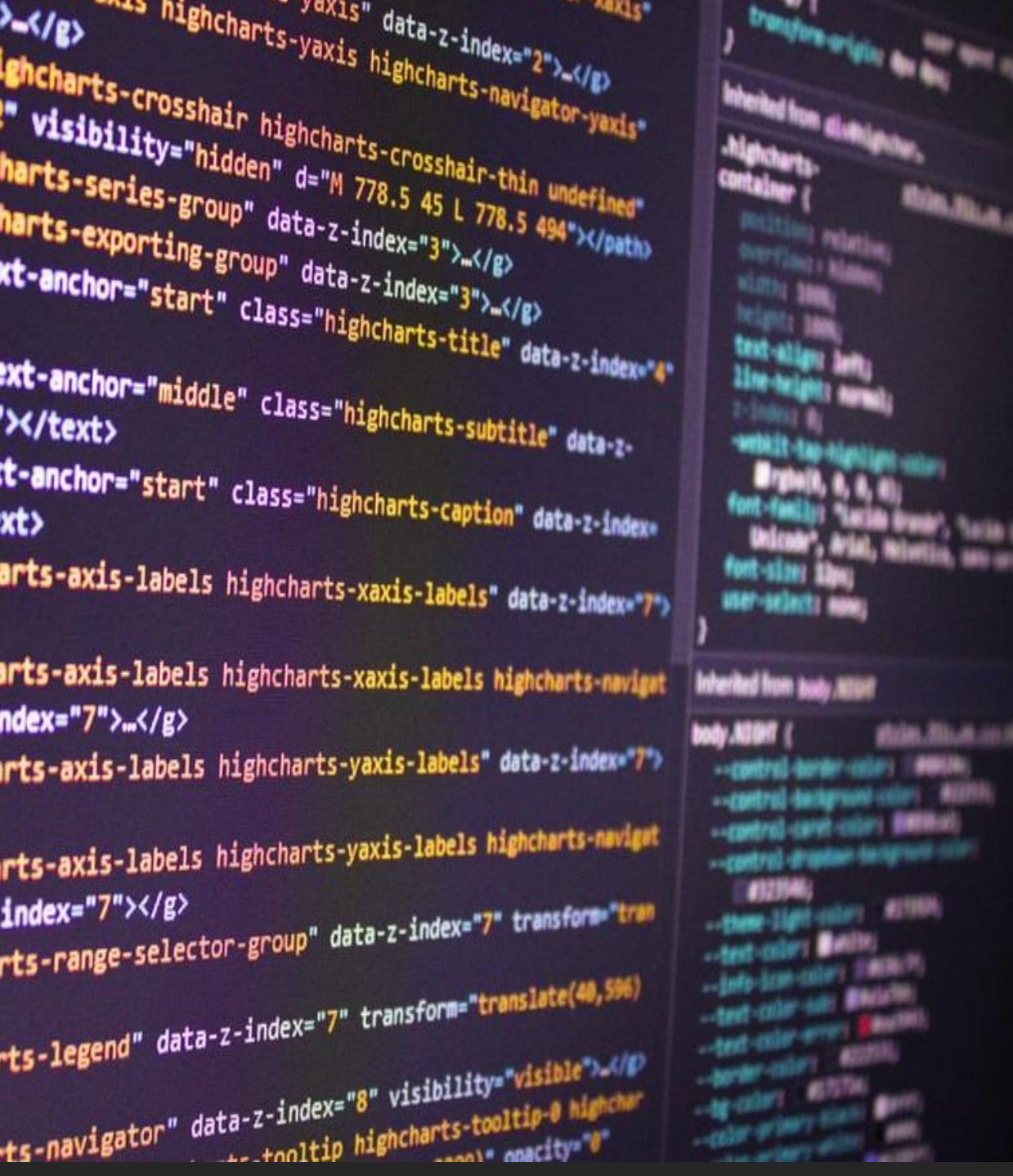




Projeto Integrador II

PROF. DR. FERNANDO T. FERNANDES

Desafio

Uma **fintech** precisa realizar o atendimento de clientes em seus canais de atendimento online.

Os clientes entram em uma fila online e são atendidos um por vez pelo atendente que gerencia a fila.

Problemas

- ☐ Quem será atendido primeiro ?
- ☐ Como atender os clientes ?
- ☐ Como atender até a fila acabar ?
- ☐ Como reaproveitar funcionalidades desenvolvidas ?
- ☐ Como solicitar ao caixa se deseja atender o próximo cliente?
- ☐ Como representar tudo isso usando apenas HTML e Javascript?

Vetores (Arrays)

Vetores (Arrays)

Posição 0 Posição 1 Posição 2



```
const fila = ["Luiz", "Ana", "Roberta"];
```

Como remover o primeiro cliente da fila ?

Operações em Arrays

push() – Adiciona um novo valor ao fim do array

```
const marcas = ["Audi", "BMW"];  
marcas.push("Fiat");
```

pop() – Remove o último elemento do array

```
const marcas = ["Audi", "BMW", "Fiat"];  
marcas.pop();  
//marcas = ["Audi", "BMW"]
```

shift() – Remove o primeiro elemento do array

```
const marcas = ["Audi", "BMW", "Fiat"];  
marcas.shift();  
//marcas = ["BMW", "Fiat"]
```

at(index) ou vetor[i] – Recupera elemento no índice informado.

Similar ao uso de colchetes. Ex: `marcas[0]` é igual a `marcas.at(0)`.



Desafio - Fila de Atendimento – Parte 1

- 1) Crie um arquivo chamado atendimento.js
- 2) Exiba inicialmente uma lista de clientes (Luiz, Ana, Roberta)
- 3) Faça o atendimento do primeiro cliente.
- 4) Reexiba a fila e mostre o próximo cliente



Tempo Estimado: **10 minutos**

Problemas

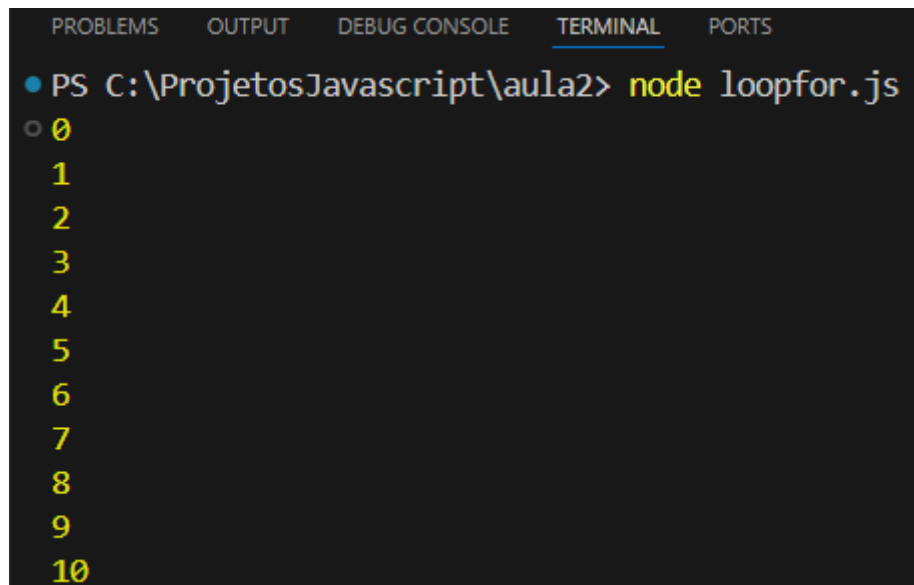
- ✓ Quem será atendido primeiro ?
- ✓ Como atender os clientes ?
- ☐ Como atender até a fila acabar ?
- ☐ Como reaproveitar funcionalidades desenvolvidas ?
- ☐ Como solicitar ao caixa se deseja atender o próximo cliente?
- ☐ Como representar tudo isso usando apenas HTML e Javascript?

Estruturas de Repetição

Loop for

Define-se o início, a condição de saída e o incremento.

```
for (let i = 0; i <= 10; i++) {  
    console.log(i);  
}
```



The screenshot shows a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active and underlined), and 'PORTS'. Below the tabs, the command prompt shows the command 'node loopfor.js' being executed. The output of the program is a list of numbers from 0 to 10, each on a new line, displayed in a yellow font.

```
PS C:\ProjetosJavascript\aula2> node loopfor.js  
0  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

Loop For..Of

Permite percorrer valores sem se preocupar com início e término.

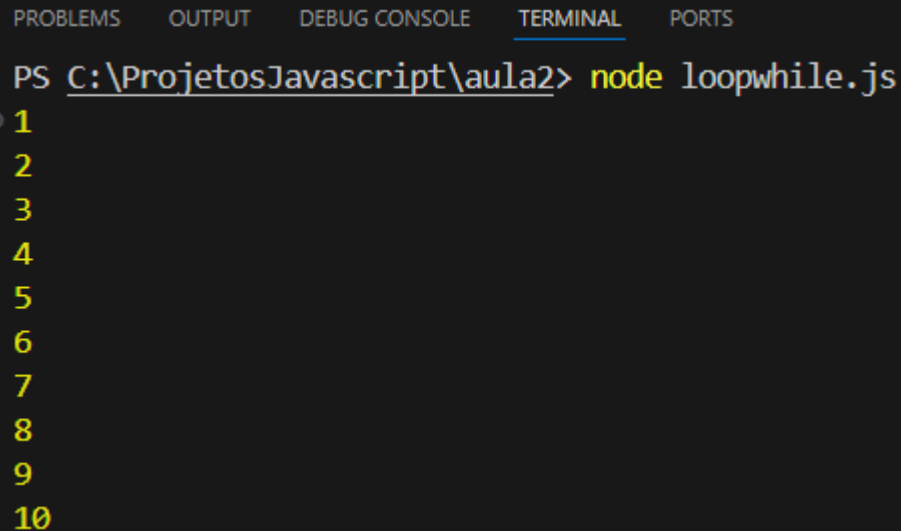
```
for (variavel of itemIterar){  
    //Codigo a executar  
}
```

```
const frutas = ["Banana", "Morango", "Abacaxi"];  
for(fruta of frutas){  
    console.log(fruta);  
}
```

Loop While

Executa um bloco de código enquanto a condição a ser testada for verdadeira.

```
let i = 1;
while(i<=10){
    console.log(i);
    i++;
}
```



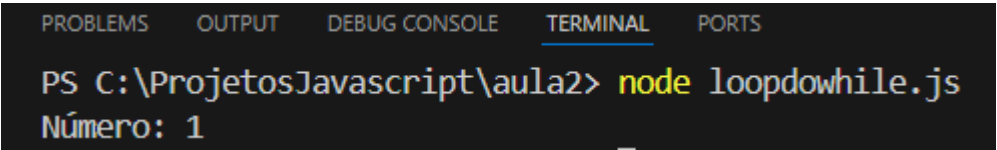
The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The command prompt shows the command `node loopwhile.js` being executed in the directory `C:\ProjetosJavascript\aula2`. The output of the program is a list of numbers from 1 to 10, each on a new line, displayed in yellow text on a dark background.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\ProjetosJavascript\aula2> node loopwhile.js
1
2
3
4
5
6
7
8
9
10
```

Do..while

Executa um bloco de código pelo menos uma vez

```
let i = 1; // Inicializa o contador
do {
  console.log("Número: " + i);
  i++;
} while (i < 2);
```



A screenshot of a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. Below the tabs, the terminal shows a command prompt 'PS C:\ProjetosJavascript\aula2>' followed by the command 'node loopdowhile.js'. The output of the command is 'Número: 1'.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\ProjetosJavascript\aula2> node loopdowhile.js
Número: 1
```



Desafio - Fila de Atendimento – Parte 2

- 1) Altere o arquivo chamado atendimento.js
- 2) Crie um loop para atender todos os clientes da fila
- 3) Exiba cada cliente que está sendo atendido



Tempo Estimado: **10 minutos**

Problemas

- ✓ Quem será atendido primeiro ?
- ✓ Como atender os clientes ?
- ✓ Como atender até a fila acabar ?
- ☐ Como reaproveitar funcionalidades desenvolvidas ?
- ☐ Como solicitar ao caixa se deseja atender o próximo cliente?
- ☐ Como representar tudo isso usando apenas HTML e Javascript?

Funções

Funções

Permitem reutilizar um trecho de código.

```
function nomeFuncao(parametro1, parametro2){  
    return parametro1 * parametro2;  
}
```

```
function soma(numero1, numero2){  
    return numero1 + numero2;  
}  
  
console.log(soma(5,6));  
console.log(soma(11,8));
```

```
C:\ProjetosJavascript\aula3>node funcoes.js  
15
```



Desafio - Fila de Atendimento – Parte 3

- 1) Altere o arquivo chamado atendimento.js
- 2) Crie uma função para **exibir a fila de atendimento** após atender um usuário
- 3) A função deverá verificar se a fila está vazia.
 - 1) Se estiver imprimir “fila vazia”
 - 2) Se não, imprimir a fila completa.
- 4) Chame a função de exibição de fila após cada atendimento



Tempo Estimado: **10 minutos**

Problemas

- ✓ Quem será atendido primeiro ?
- ✓ Como atender os clientes ?
- ✓ Como atender até a fila acabar ?
- ✓ Como reaproveitar funcionalidades desenvolvidas ?
- ☐ Como solicitar ao caixa se deseja atender o próximo cliente?
- ☐ Como representar tudo isso usando apenas HTML e Javascript?

Interagindo com o usuário

Opções de interação em javascript

Permitem exibir uma caixa de diálogo simples ao usuário.

```
alert("Bem-vindo à nossa página!");
```



Caixa de diálogo
simples

```
let resposta = confirm("Você deseja continuar?");
```



Caixa de diálogo
com opções

```
let nome = prompt("Qual é o seu nome?");
```



Solicitar a entrada
do usuário

atendimento.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Minha Fintech</title>
  <script src="atendimento.js"></script>
</head>
<body>
  <h1>Atendimento</h1>

  <script>
    //Crio a variável para armazenar a fila de clientes
    let fila = [];
  </script>

  <!-- Vincular os botões às funções JavaScript -->

  <button onclick="adicionarCliente()">Adicionar Cliente</button>
  <button onclick="atenderCliente()">Atender Cliente</button>

</body>
</html>
```

atendimento.js

```
function adicionarCliente() {
    let nomeCliente = prompt('Digite o nome do cliente:');
    if(nomeCliente) {
        fila.push(nomeCliente);
    }
}

function atenderCliente() {
    if(fila.length > 0) {
        let clienteAtendido = fila.shift();
        alert('Atendendo o cliente: ' + clienteAtendido);
    } else {
        alert('Fila vazia!');
    }
}
```

Atendimento

Adicionar Cliente

Atender Cliente



Desafio - Fila de Atendimento – Parte 4

- 1) Abra o arquivo chamado `atendimento.js`
- 2) Adicione uma função para exibir a fila de atendimento após atender um usuário
- 3) Após o atendimento de cada cliente, chame a função para exibir a fila.
- 4) Ao atender um cliente, pergunte ao caixa se deseja atender o próximo cliente.
 - 1) Se o caixa responder ok, atenda o próximo cliente
 - 2) Senão, encerre o atendimento
- 5) Crie um repositório chamado **“fintechjs”** no github
- 6) Suba a sua primeira versão estável do projeto (Branch - *main*)



Tempo Estimado: **10 minutos**

Arrow Functions

Permitem criar funções com uma sintaxe mais enxuta!

```
const saudacao = (nome) => {  
  return `Olá, ${nome}!`;  
};  
  
console.log(saudacao("Ana"));
```

O sinal de **seta** (**=>**) é usado para definir a instrução da função e o valor que ela irá retornar.

Não precisa usar a palavra-chave function!

Mesmas funções escritas
de forma diferente

```
function saudacao(nome){  
  return `Olá, ${nome}!`;  
}  
  
console.log(saudacao("Ana"));
```

Arrow Functions

Se a função não requer parâmetros, coloca-se parênteses vazios

```
const diaDoMes = () => new Date().getDate();  
console.log(diaDoMes());
```

```
const horaDoDia = () => new Date().getHours();  
console.log(horaDoDia());
```

```
hello = () => console.log("Hello World!");  
hello();
```

```
const somar = a => a + 2;  
console.log(somar(2));
```

Se a função tem apenas um parâmetro,
não precisamos coloca-lo entre parênteses



Obs: Quando temos apenas uma instrução, não precisamos escrever a palavra **return**

Exemplo – saudacao.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Arrow Function</title>
</head>

<body>
  <h1>Arrow Function</h1>
  <p>Veja como é simples usar uma arrow function (função e seta)</p>

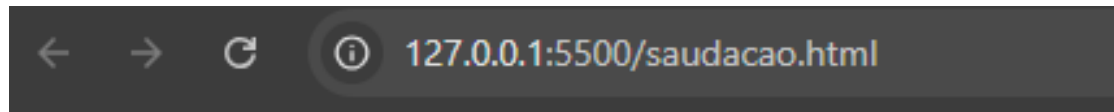
  <script>
    saudacao = () => {
      alert("Bem-vindos às Arrow Functions!");
    }

    despedida = () => alert("Até logo!");
  </script>

  <p>Clique nos botões abaixo:</p>

  <input type="button" name="input-hello" id="input-hello" onclick="saudacao()" value="Olá!" />
  <input type="button" name="input-bye" id="input-bye" onclick="despedida()" value="Adeus!" />
</body>
</html>
```

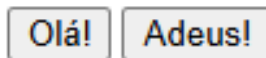
Exemplo – saudacao.html



Arrow Function

Veja como é simples usar uma arrow function (função e seta)

Clique nos botões abaixo:





Desafio - Arrow Function - Saudação

Autoatendimento online

A Fintech viu potencial em seu trabalho e pediu que você desenvolva uma nova tela para autoatendimento de seus clientes. Para isso, você deve:

1. Criar uma nova *Branch* chamada “autoatendimento”
 1. `git checkout -b autoatendimento`
2. Criar uma página HTML para o autoatendimento (autoatendimento.html)
3. Criar um arquivo JS para o autoatendimento (autoatendimento.js)
4. Ao carregar a página HTML, dê as boas-vindas ao cliente com base na hora do dia
5. Crie uma **arrow function** chamada **saudacao** no arquivo autoatendimento.js
6. Chame a função no corpo da página HTML



Tempo Estimado: **10 minutos**



Desafio Final - Autoatendimento

Autoatendimento online

1. Na página HTML crie os botões “Consultar saldo”, “Depositar” e “Sacar”.
2. No arquivo JS:
 1. Defina o saldo inicial do cliente como 1000;
 2. Use uma *arrow function* para a função de saudação
 3. **Para depositar, converta o valor digitado para número antes de somar ao saldo**
 1. `let valor = Number(prompt("Digite o valor do saque:"));`
3. Use outras funções ou arrow functions para as demais funcionalidades.
4. Ao terminar, suba a Branch ao github e solicite o pull request



Tempo Estimado: **15 minutos**

Problemas

- ✓ Quem será atendido primeiro ?
- ✓ Como atender os clientes ?
- ✓ Como atender até a fila acabar ?
- ✓ Como reaproveitar funcionalidades desenvolvidas ?
- ✓ Como solicitar ao caixa se deseja atender o próximo cliente?
- ✓ Como representar tudo isso usando apenas HTML e Javascript?

Obrigado!



proffernandotfernandes